Chapter 2

The Simplex Method

The Simplex Method is "a systematic procedure for generating and testing candidate vertex solutions to a linear program." (Gill, Murray, and Wright, p. 337) It begins at an arbitrary corner of the solution set. At each iteration, the Simplex Method selects the variable that will produce the largest change towards the minimum (or maximum) solution. That variable replaces one of its compatriots that is most severely restricting it, thus moving the Simplex Method to a different corner of the solution set and closer to the final solution. In addition, the Simplex Method can determine if no solution actually exists. Note that the algorithm is greedy since it selects the best choice at each iteration without needing information from previous or future iterations.

The Simplex Method solves a linear program of the form described in Figure 3. Here, the coefficients c_j represent the respective weights, or costs, of the variables x_j. The minimized statement is similarly called the cost of the solution. The coefficients of the system of equations are represented by a_{ij}, and any constant values in the system of equations are combined on the right-hand side of the inequality in the variables b_i. Combined, these statements represent a linear program, to which we seek a solution of minimum cost.

Figure 3

$$\begin{aligned} \text{minimize} \quad & \sum_{j=1}^{n} c_j x_j \\ \text{subject to} \quad & \sum a_{ij} x_j \le b_i \quad (i = 1,2,\ldots,m < n) \\ & x_j \ge 0 \quad (j = 1,2,\ldots,n) \end{aligned}$$

A Linear Program

Solving this linear program involves solutions of the set of equations. If no solution to the set of equations is yet known, slack variables $x_{n+1}, x_{n+2}, \ldots, x_{n+m}$, adding no cost to the solution, are introduced. The initial basic feasible solution (BFS) will be the solution of the linear program where the following holds:

$$\begin{aligned} x_i &= 0 \quad (i = 1,2,\cdots,n) \\ x_i &= b_{n-i} \quad (i = n+1, n+2, \cdots, n+m) \end{aligned}$$

Once a solution to the linear program has been found, successive improvements are made to the solution. In particular, one of the nonbasic variables (with a value of zero) is chosen to be increased so that the value of the cost function, $\sum_{j=1}^{n} c_j x_j$, decreases. That variable is then increased, maintaining the equality of all the equations while keeping the other nonbasic variables at zero, until one of the basic (nonzero) variables is reduced to zero and thus removed from the basis. At this point, a new solution has been determined at a different corner of the solution set.

The process is then repeated with a new variable becoming basic as another becomes nonbasic. Eventually, one of three things will happen. First, a solution may occur where no nonbasic variable will decrease the cost, in which case the current solution is the optimal solution. Second, a non-basic variable might increase to infinity without causing a basic-variable to become zero, resulting in an unbounded solution. Finally, no solution may actually exist and the Simplex Method must abort. As is common for research in linear programming, the possibility that the Simplex Method might return to a previously visited corner will not be considered here.

The primary data structure used by the Simplex Method is "sometimes called a dictionary, since the values of the basic variables may be computed ('looked up') by choosing values for the nonbasic variables." (Gill, Murray, and Wright, p. 337) Dictionaries contain a representation of the set of equations appropriately adjusted to the current basis. The use of dictionaries provide an intuitive understanding of why each variable enters and leaves the basis. The drawback to dictionaries, however, is the necessary step of updating them which can be time-consuming. Computer implementation is possible, but a version of the Simplex Method has evolved with a more efficient matrix-oriented approach to the same problem. This new implementation became known as the Revised Simplex Method.

The Revised Simplex Method

The Revised Simplex Method describes linear programs as matrix entities and presents the Simplex Method as a series of linear algebra computations. Instead of spending time updating dictionaries at the end of each iteration, the Revised Simplex Method does its heavy calculation at the beginning of each iteration, resulting in much less at the end. As Chvátal observed,

> "Each iteration of the revised simplex method may or may not take less time than the corresponding iteration of the standard simplex method. The outcome of this comparison depends not only on the particular implementation of the the revised simplex method but also on the nature of the data. ...on the large and sparse [linear programming] problems solved in applications, the revised simplex method works faster than the standard simplex method. This is the reason why modern computer programs for solving [linear programming] problems always use some form of the revised simplex method." (pp. 97-98)

First of all, a linear program needs to be expressed in terms of matrices. for instance, the linear program stated previously:

$$\begin{aligned} \text{minimize} \quad & \sum_{j=1}^{n} c_j x_j \\ \text{subject to} \quad & \sum a_{ij} x_j \le b_i \quad (i = 1,2,\ldots,m) \\ & x_j \ge 0 \quad (j = 1,2,\ldots,n) \end{aligned}$$

can instead be recast in the form:

$$\begin{aligned} \text{minimize} \quad & cx \\ \text{subject to} \quad & Ax \le b \\ & x \ge 0 \end{aligned}$$

where:

$$c = \begin{bmatrix} c_1 & c_2 & \cdots & c_n \end{bmatrix}$$

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{11} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{n2} & \cdots & a_{mn} \end{bmatrix}$$

$$b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

The steps of the Simplex Method also need to be expressed in the matrix format of the Revised Simplex Method. The basis matrix, *B*, consists of the column entries of *A* corresponding to the coefficients of the variables currently in the basis. That is if x_2 is the fourth entry of the basis, then $\begin{bmatrix} a_{12} & a_{22} & \cdots & a_{m2} \end{bmatrix}^T$ is the fourth column of *B*. (Note that *B* is therefore an $m \times m$ matrix.) The non-basic columns of *A* constitute a similar though likely not square, matrix referred to here as *V*.

Simplex Method	Revised Simplex Method
Determine the current basis, *d*.	$d = B^{-1}b$
Choose x_j to enter the basis based on the greatest cost contribution.	$\tilde{c} = c_V - c_B \cdot B^{-1} \cdot V$ $\left\{ j \middle\vert \tilde{c}_j = \min_t (\tilde{c}_t) \right\}$
If x_j cannot decrease the cost, *d* is optimal solution.	If $\tilde{c}_j \ge 0$, *d* is optimal solution.
Determine x_i that first exits the basis (becomes zero) as x_j increases.	$w = B^{-1} \cdot A_j$ $\left\{ i \middle\vert \frac{d_i}{w_i} = \min_t \left(\frac{d_t}{w_t} \right), w_t > 0 \right\}$
If x_i can decrease without causing another variable to leave the basis, the solution is unbounded.	If $w_i \le 0$ for all *i*, the solution is unbounded.
Update dictionary.	Update B^{-1}.

The physical limitations of a computer can become a factor. Round-off error and significant digit loss are common problems in matrix manipulations, especially when the entries of a matrix have widely differing orders of magnitude. Such matrices are usually not well-conditioned. Matrices that are even close to ill-conditioned can cause severe round-off errors in the finite memory of a computer. Implementation of the Revised Simplex Method becomes more than simply programming the algorithm; it also becomes a task in numerical stability.

Algorithmic analysis of the Revised Simplex Method shows that the costliest step is the inversion of the basis, taking $m^2(m-1)$ multiplications and $m(m-1)$ additions, a total of $m^3 - m$ floating-point (real number) calculations. Many variants of the Revised Simplex Method have been designed to

reduce this $O(m^3)$-time algorithm as well as improve its accuracy.

The Bartels-Golub Method

The first variant of the Revised Simplex Method was the Bartels-Golub Method which mildly improved running time and storage space and significantly improved numerical accuracy. The Bartels-Golub Method avoids the problem of matrix-inversion by simply avoiding matrix-inversion altogether. Instead, the basis is decomposed into upper- (*U*) and lower- (*L*) triangular factors, which only takes about $\frac{1}{3}m^3$ computations to determine. In addition, LU-decomposition of a sparse basis produces sparse factors, preserving some space lost to dense inverse-bases. Best of all, the factors can be permuted via pivoting to decrease the round-off errors due to a near-singular basis.

The actual implementation of the Bartels-Golub Method closely parallels the Revised Simplex Method, utilizing the *L* and *U* factors where the inverse basis was required before. The increase in speed was impressive, but the improvement in accuracy was dramatic, occasionally solving problems that the Revised Simplex Method simply could not do before.

Revised Simplex Method	Bartels-Golub Method
$d = B^{-1}b$	$d = U^{-1} \cdot L^{-1} \cdot b$
$\tilde{c} = c_N - c_B \cdot B^{-1} \cdot N$ $\left\{ j \middle\| \tilde{c}_j = \min_t(\tilde{c}_t) \right\}$	$\tilde{c} = c_N - c_B \cdot U^{-1} \cdot L^{-1} \cdot N$ $\left\{ j \middle\| \tilde{c}_j = \min_t(\tilde{c}_t) \right\}$
If $\tilde{c}_j \geq 0$, *d* is optimal solution.	If $\tilde{c}_j \geq 0$, *d* is optimal solution.
$w = B^{-1} \cdot A_j$ $\left\{ i \middle\| \frac{d_i}{w_i} = \min_t \left(\frac{d_t}{w_t} \right), w_t > 0 \right\}$	$w = U^{-1} \cdot L^{-1} \cdot A_j$ $\left\{ i \middle\| \frac{d_i}{w_i} = \min_t \left(\frac{d_t}{w_t} \right), w_t > 0 \right\}$
If $w_i \leq 0$ for all *i*, the solution is unbounded.	If $w_i \leq 0$ for all *i*, the solution is unbounded.
Update B^{-1}.	Update U^{-1} and L^{-1}.

Where the Revised Simplex Method called for the inverse of the basis to be multiplied, the Bartels-Golub substitutes two backsolves involving the upper and lower triangular basis factors. When a variable is selected to enter the basis, the Bartels-Golub Method maintains that *L* not be changed, therefore requiring that *U* must make the changes necessary to accomodate the new variable. Since the variable brings an entire column of coefficients into the basis, an entire column of *U* will change when the new variable enters, creating a "spike" in the upper triangular formation.

The various implementations and variations of the Bartels-Golub generally diverge with the next step: reduction of the spiked upper triangular matrix back to an upper-triangular matrix. As Chvátal noted, "some device is used to facilitate their solutions and is updated at the end of each iteration." (p. 105) The most common methods attempt to amortize the cost of refactorization to reduce the average asymptotic complexity of the Revised Simplex Method. These methods have met with varying levels of success prompting this study on their effectiveness.

The Sparse Bartels-Golub Method

Despite the improvements that the Bartels-Golub Method made to the Revised Simplex Method, it still failed to significantly improve the asymptotic complexity. Further variations attempted to amortize the cost of LU-decomposition. If some method, efficient in the short-term, could be found to update the basis-inverse without necessitating a full decomposition of the basis, then perhaps the algorithm's overall complexity could be reduced. If such a method could forestall full LU-decomposition to less than once every *m* iterations, then the overall time-complexity of the algorithm could be reduced to $O(m^2)$ per iteration.

John Reid proposed such an amortization directly on the Bartels-Golub Method. Instead of decomposing the basis with each iteration, he suggested that the lower-triangular matrix be stored in memory as a product of lower-triangular eta matrices, matrices that differed from the identity matrix in a single row or column. Such a formation drastically simplifies many algebraic manipulations, for the inverse of an upper- or lower-triangular eta matrix

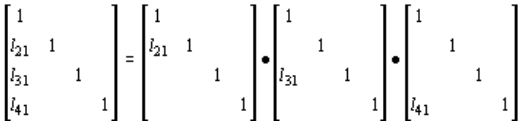
is the very same eta matrix with the off-diagonal elements negated. Therefore, if L is expressed as a product of eta matrices, then the inverse of L is the product of those same eta matrices -- in reverse order with the off-diagonal elements negated.

As it turns out, decomposing a lower triangular matrix into a product of eta matrices does not involve any computations at all. Each column of the lower triangular matrix is simply isolated in its own eta matrix. If the matrices are multipled left-to-right with the columns in order from right-to-left as in Figure 4 then the original matrix can be recreated. Furthermore, these column-eta matrices can be reduced into single-entry column eta matrices by just separating the column entries as in Figure 5.

Figure 4

Decomposition of a lower-triangular matrix into column-eta matrices.

Figure 5



Decomposition of a lower-triangular column-eta matrix into single-entry eta matrices.

In each iteration of the Bartels-Golub Method, a "spike" occurs in the upper-triangular factor, U, due to the coefficients of the variable entering the basis. Where w indicates the incoming jth column and e_j^T is the jth column of the identity matrix, we have:

$$A = L \cdot U$$
$$A + w \cdot e_j^T = L \cdot U + w \cdot e_j^T$$

If we keep L constant and allow only U to change, we discover that

$$\begin{aligned} A + w \cdot e_j^T &= L \cdot U + w \cdot e_j^T \\ &= L \cdot \left(U + L^{-1} \cdot w \cdot e_j^T \right) \\ &= L \cdot \tilde{U} \end{aligned}$$

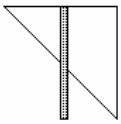
Here we can tell that U differs from $\tilde{U}$ only in the jth column. The non-zero elements below the diagonal in this column create the charactersitic spike of the Bartels-Golub Method.

Bartels-Golub Method	Sparse Bartels-Golub Method
$d = U^{-1} \cdot L^{-1} \cdot b$	$d = U^{-1} \cdot \prod_t \eta_t \cdot b$
$\tilde{c} = c_N - c_B \cdot U^{-1} \cdot L^{-1} \cdot V$ $\left\{ j \middle\| \tilde{c}_j = \min_t (\tilde{c}_t) \right\}$	$\tilde{c} = c_N - c_B \cdot U^{-1} \cdot \prod_t \eta_t \cdot V$ $\left\{ j \middle\| \tilde{c}_j = \min_t (\tilde{c}_t) \right\}$
If $\tilde{c}_j \geq 0$, d is optimal solution.	If $\tilde{c}_j \geq 0$, d is optimal solution.
$w = U^{-1} \cdot L^{-1} \cdot A_j$ $\left\{ i \middle\| \frac{d_i}{w_i} = \min_t \left(\frac{d_t}{w_t} \right), w_t > 0 \right\}$	$w = U^{-1} \cdot \prod_t \eta_t \cdot A_j$ $\left\{ i \middle\| \frac{d_i}{w_i} = \min_t \left(\frac{d_t}{w_t} \right), w_t > 0 \right\}$
If $w_i \leq 0$ for all i, the solution is unbounded.	If $w_i \leq 0$ for all i, the solution is unbounded.

Update U^{-1} and L^{-1}.	Update U^{-1} and create any necessary eta matrices. If there are too many eta matrices, completely refactor the basis.

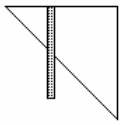
Rather than completely refactoring the basis, Reid suggested applying LU-decomposition only to the part of $\tilde{U}$ that remained upper-Hessenberg after the lowest element of the spike was column-permuted back to the diagonal (see Figures 6, 7, and 8). The resulting lower-triangular submatrix can then be easily factored into eta matrices and joined with the current eta factors of *L* without further calculations. Meanwhile, the resulting upper-triangular factor remains a part of our new *U* with necessary calculations to row elements outside the factored region.

Figure 6



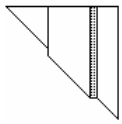
An incoming basic column creates a spike in the upper-triangular factor.

Figure 7



The spike need not extend to the botom row.

Figure 8



When permuted, the matrix becomes partially upper-Hessenberg.

In comparison, the Sparse Bartels-Golub Method is no more complex than the Bartels-Golub Method, using LU-decomposition and matrix algebra. However, instead of just *L* and *U*, the factors become the lower-triangular eta matrices and *U*. In the implementation used here, the eta matrices were reduced to single-entry eta matrices. Instead of having to store the entire matrix, it is only necessary to store the location and value of the off-diagonal element for each matrix.

Eventually, the number of eta matrices will become so large that it becomes cheaper to decompose the basis from scratch than continue using the current data structures. This is the step that must occur infrequently. Also, such a refactorization may occur prematurely in an attempt to promote stability if noticeable round-off errors begin to occur. So long as the refactorizations occur less than once every *m* times, though, the amortized cost of the Sparse Bartels-Golub Method improves significantly to $O(m^2)$. Practical use has shown, though, that "in solving large sparse problems, the basis is refactorized quite frequently, often after every twenty iterations of so." (Chvátal, p. 111)

Disadvantages also occur as with any attempt at amortization. If the pathological, or worst-case, situation arises then the spike always occurs in the first column and extends to the bottom row, and the Sparse Bartels-Golub Method becomes worse than the Bartels-Golub Method. The upper-triangular matrix will always be fully-decomposed resulting in huge amounts of fill-in, large numbers of eta matrices, and the very $O(n^3)$-cost decomposition that was supposed to have been avoided.

The Forrest-Tomlin Method

J. J. H. Forrest and J. A. Tomlin attempted a different type of approach to maintaining the upper-triangular characteristic of *U*. The Forrest-Tomlin Method takes the row with the same diagonal entry as the spike and calculates the matrix factor that algebraically cancels each row entry to zero. Thus, the only non-zero entry on the diagonal -- due to the spike -- can be permuted to the bottom right corner of the matrix. The spiked column becomes the right-most column and the other columns and rows permute one up and one left respectively leaving *U* again upper-triangular.

Calculation of the matrix factors involves a backsolve with *U* against the row entries that are to be eliminated. Thus the matrix factors are actually a series of row-eta matrices, and, like the eta matrices of the Sparse Bartels-Golub Method, it is the number of these factors that will determine when the next complete LU-decomposition should occur.

Bartels-Golub Method	Forrest-Tomlin Method
$d = U^{-1} \cdot L^{-1} \cdot b$	$d = U^{-1} \cdot \prod_t R_t \cdot L^{-1} \cdot b$
$\tilde{c} = c_N - c_B \cdot U^{-1} \cdot L^{-1} \cdot N$ $\left\{ j \middle\| \tilde{c}_j = \min_t(\tilde{c}_t) \right\}$	$\tilde{c} = c_N - c_B \cdot U^{-1} \cdot \prod_t R_t \cdot L^{-1} \cdot N$ $\left\{ j \middle\| \tilde{c}_j = \min_t(\tilde{c}_t) \right\}$
If $\tilde{c}_j \geq 0$, *d* is optimal solution.	If $\tilde{c}_j \geq 0$, *d* is optimal solution.
$w = U^{-1} \cdot L^{-1} \cdot A_j$	$w = U^{-1} \cdot \prod_t R_t \cdot L^{-1} \cdot A_j$

$\left\{i \middle\| \frac{d_i}{w_i} = \min_t \left(\frac{d_t}{w_t}\right), w_t > 0\right\}$	$\left\{i \middle\| \frac{d_i}{w_i} = \min_t \left(\frac{d_t}{w_t}\right), w_t > 0\right\}$
If $w_i \leq 0$ for all i, the solution is unbounded.	If $w_i \leq 0$ for all i, the solution is unbounded.
Update U^{-1} and L^{-1}.	Update U^{-1}, creating a row factor as necessary. If there are too many factors, completely refactor the basis.

The Forrest-Tomlin Method has some significant advantages over the Sparse Bartels-Golub Method, most notably in the predictable structure of the non-zero factors. At most one row-eta matrix factor will occur for each iteration where an unpredicatable number occured before. The code can take advantage of such knowledge for predicting necessary storage space and calculations. Fill-in should also be relatively slow, since fill-in can only occur within the spiked column.

For that extra ease, though, the Forrest-Tomlin Method does pay a price. Where the Sparse Bartels-Golub Method allowed LU-decomposition to pivot for numerical stability, the Forrest-Tomlin Method makes no such allowances. Therefore, severe calculation errors due to near-singular matrices are more likely to occur.

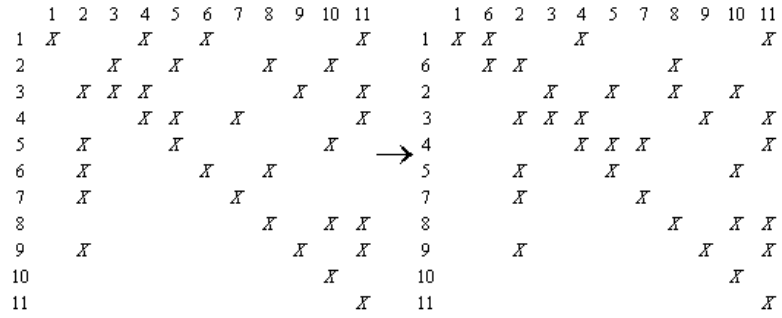
Reid's Method

Years later, Reid reexamined his Sparse Bartels-Golub Method and declared an improvement that could vastly improve its performance. When the Sparse Bartels-Golub Method attempts to LU-decompose the spike, it is taking a square sub-matrix of *U* on the diagonal with the spike in the left-most column. This he termed the "bump" and he realized that any and all fill-in that must occur in *U* will occur in the rows spanned by the bump.

The task, then, is to find a way to reduce that bump before attempting to decompose it. As it turns out, this is a simple task. Reid noted that any row of the bump that has only one non-zero entry, which he termed a *row singleton*, could be permuted, or rotated, to place the non-zero entry in the bottom right corner of the bump as illustrated in Figure 9. This allowed the bump to slough off the bottom row and right column for they were now upper-triangular as intended. Similarly, as shown in Figure 10, any column other than the spike with only one non-zero entry, a *column singleton*, could be rotated to place its entry in the upper left corner of the bump, eliminating the top row and left column from the bump.

Eventually, one of two things must occur. Either the bump will become non-reducible or it will be permuted until it is completely upper triangular. If a bump remains it is then LU-decomposed, but note that since the bump now spans fewer rows there will very likely be much less fill-in than if the rotations had never occurred. On the other hand, if it is at all possible to eliminate the bump purely on the basis of rotations, Reid's Method will do so and completely avoid fill-in altogether.

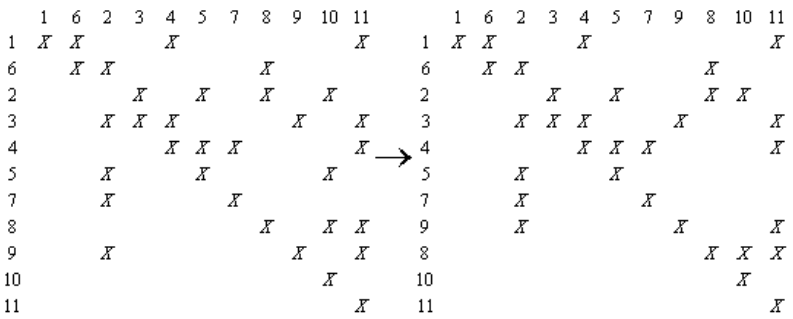
Figure 9



Column Rotation

Before the rotation, the bump consists of rows and columns 2 through 9. Within the bump, column 6 has only one entry, therefore it is a column singleton and can be rotated out of the bump by permuting that entry to the upper left corner of the bump. After the rotation, the bump is reduced by one row and one column.

Figure 10



Row Rotation

Continuing from the previous example, row 8 has only one entry inside the bump. This entry is a row singleton that can be permuted to the lower right corner of the bump. After the rotation, the bump again is reduced by one row and one column.

The rotation step occurs during the update step of the Sparse Bartels-Golub Method after the spike enters U and before the LU decomposition occurs. Permutation matrices representing the rotations in U need to be accounted for, but otherwise Reid's Method is identical to the Sparse Bartels-Golub Method.

Reid's Method significantly reduces the growth of the number of eta matrices in the Sparse Bartels-Golub Method, and the basis should not need to be decomposed nearly as often. The use of LU-decomposition on any remaining bump still allows some attempt to maintain stability. Realize, though, that the rotations make absolutely no allowance for stability whatsoever, so Reid's Method remains numerically less stable than the Sparse Bartels-Golub Method.